

Zero to Hero Study Handbook: Production CrewAI Multi-Agent Platform

This handbook is built from static analysis of this repository's files. It is designed to help a new learner go from first contact to contributor-level understanding.

Module 1: Foundations & Architecture

1) What this project does

This project is a production-oriented multi-agent AI collaboration platform.

Main use cases implemented in code: - Plan complex objectives into dependency-aware task graphs. - Run specialized role-based agents across those tasks. - Use tools (search, calculators, file readers, semantic memory, etc.) during execution. - Verify outputs (fact/QA/reflection confidence), optionally run consensus reasoning. - Generate report artifacts in Markdown/JSON and export to HTML/PDF. - Expose everything through FastAPI, Typer CLI, and Streamlit dashboard.

Primary runtime surfaces: - API: `src/crew_platform/api/main.py` - CLI: `src/crew_platform/cli/main.py` - Streamlit launcher: `app.py` and `streamlit_app/` - Core orchestration: `src/crew_platform/orchestration/`

2) Core paradigms and patterns used here

Definitions first, then where each appears.

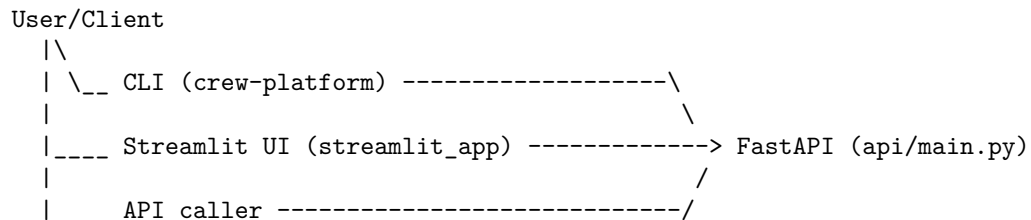
- Schema-first design:
 - Definition: use typed schemas as the contract for inputs/outputs.
 - Used via Pydantic models in `src/crew_platform/config/models.py`, `src/crew_platform/orchestration/models.py`, and tool input/output models.
- Service-oriented orchestration:
 - Definition: a central service composes smaller subsystems.
 - `CollaborationService` (`src/crew_platform/orchestration/service.py`) composes planner, executor, verifier, consensus, report generator, memory, persistence, and tools.
- DAG-based task execution:
 - Definition: tasks have explicit dependencies; ready tasks run when dependencies are complete.
 - Implemented in `DAGExecutor.execute()` (`src/crew_platform/orchestration/executor.py`).
- Async I/O:
 - Definition: network and orchestration operations are `async` to improve responsiveness.
 - Seen in API handlers, `OllamaProvider`, tool execution, MCP client, and orchestration services.
- Graceful degradation:

- Definition: failure in optional subsystems should not crash the primary workflow.
- Examples: telemetry disable in `crewai_runner.py`, MLflow best-effort logging, Chroma disable fallback in `memory/runtime.py`, network offline fallback in tool modules.
- Registry + factory pattern for tools:
 - Definition: tools are registered dynamically and invoked by name through a common interface.
 - `ToolRegistry` in `tools/registry.py`, constructed via `create_default_registry()` in `tools/factory.py`.
- Plugin-style extension:
 - Definition: external YAML manifests can add agents without modifying core orchestration code.
 - `PluginLoader` + `_load_marketplace_agents()` in `service.py`.

3) Architecture overview

Major components and interaction: - Config + agent catalog loaders: - `load_settings()` and `load_agent_catalog()` in `src/crew_platform/config/models.py`.
 - Orchestration core: - `PlannerService` -> `DAGExecutor` -> `VerificationService` -> `ConsensusService` -> `ReportGenerator`. - LLM layer: - `OllamaProvider` in `src/crew_platform/llm/ollama.py`. - Memory layer: - SQLite persistence (`memory/persistence.py`) + semantic memory (`memory/runtime.py`, `memory/chroma_store.py`). - Tooling layer: - `ToolRegistry` + many `BaseTool` implementations in `src/crew_platform/tools/`. - Interfaces: - FastAPI (`api/main.py`), CLI (`cli/main.py`), Streamlit (`streamlit_app/`). - Optional interoperability: - MCP server/client wrappers (`src/crew_platform/mcp/`).

ASCII architecture flow:



FastAPI/CLI calls `CollaborationService`

```

-> PlannerService.build_plan(query)
-> (optional human approval gate)
-> DAGExecutor.execute(...)
    -> ToolRegistry.invoke(...)
    -> OllamaProvider.generate(...)
    -> (optional) CrewAIRunner.run_task(...)
-> VerificationService.verify(tasks)
-> (optional) ConsensusService.run(...)

```

```
-> ReportGenerator.generate(...)
-> PersistenceStore.save_* + RuntimeMemory.remember_run(...)
```

Data stores:

- SQLite: artifacts/platform.db
- Chroma: artifacts/chroma/
- Reports: data/reports/*.md|*.json|*.html|*.pdf
- Logs: logs/agent_runs.jsonl

Module 2: Repository Map

Focus: files a new contributor should understand first.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Project metadata, dependencies, scripts	[project.scripts]crew-platform, crew-platform-api	requires-python, dependency list
configs/settings.yaml	Runtime configuration source of truth	N/A (YAML)	llm.*, orchestration.*, memory.*, api.*, tools.*, reports.*
configs/agents.yaml	Enterprise agent library	N/A (YAML entries)	agents[].role, goal, tools, constraints, output_schema
src/crew_platform/config/models.py	Configuration loader contracts	Settings, AgentProfile, load_settings(), load_agent_catalog()	Default values for all config sections
src/crew_platform/orchestration/models.py	Orchestration data structures	TaskSpec, TaskExecution, PlanProposal, CrewRunResult, VerificationResult	RunStatus, TaskStatus, enums
src/crew_platform/orchestration/service.py	Orchestration workflow service	CollaborationService, create_plan(), execute_run(), chat()	plan_approval_required, consensus_enabled, consensus_trigger_confidence
src/crew_platform/orchestration/planners.py	Orchestration planning + fallback DAG creation	PlannerService, _try_llm_plan(), _fallback_plan()	set_plan(llm.planner_model, default_retry_limit)

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/crew_platform/Orchestration/executor.py	Orchestration aware task execution engine	BaseExecutor, _execute_single_task	max_parallel_tasks, taskify_backoff_seconds, use_crewai_execution
src/crew_platform/Orchestration/verification.py	Post-execution ity/confidence scoring	VerificationService	sensifs(trigger_confidence threshold
src/crew_platform/Orchestration/consensus.py	Orchestration consensus reasoning	ConsensusService.run()	consensus_models
src/crew_platform/Orchestration/crew_adapter.py	Crew adaptation (optional execution path)	CrewAdapter, CrewAdapter.run()	CREWAI_*, OTEL_SDK_DISABLED, env defaults
src/crew_platform/Ollama/ollama.py	Ollama/ollama abstraction	OllamaProvider.get_chat(), embed()	base_url(), request_timeout_seconds, auto_pull_missing_models
src/crew_platform/Tools/base.py	Tools/base interface contract	BaseTool, ToolDescriptor	input_model, output_model, schema pattern
src/crew_platform/Tools/registry.py	Tools/registry invocation, telemetry	ToolRegistry.register(), invoke()	metrics/tracing keys like tool.<name>.latency_ms
src/crew_platform/Tools/factory.py	Tools/factory toolset and aliases	create_default_registry()	enabled_tools, optional_tools, enable_python_tool
src/crew_platform/Storage/persistence.py	Storage/persistence layer	PersistenceStore.fetch_*	DB path from memory.sqlite_path
src/crew_platform/Storage/memory.py	Storage/memory semantic memory manager	RuntimeMemory.search(), remember_run()	CREW, PLATFORM_DISABLE_CHROMA, memory.retrieval_top_k
src/crew_platform/RAG/ingestion.py	RAG/ingestion from files/URLs	RAGIngestionService.ingest_url()	rag_index_path(), chunk_overlap, max_chunks_per_doc
src/crew_platform/RAG/retriever.py	RAG/retriever retrieval service	RAGRetriever.retrieve()	memory.retrieval_top_k
src/crew_platform/Reports/generator.py	Reports/generator assembly/export	ReportGenerator.generate_on_demand()	reports(output_dir, ways_formats, on_demand_formats
src/crew_platform/Fapi/Fapi.py	Fapi/Fapi endpoints and request models	/crew, /tasks, /reports, /knowledge, /mcp/* handlers	API host/port from settings

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/crew_platform/api/main.py	CLI commands	run, agents, task, report, memory, doctor, dashboard	--api-url, --force-consensus
streamlit_app/	UI pages calling API endpoints	Home.py + page scripts	CREW_PLATFORM_API_URL
scripts/run_demo.py	Programmatic demo flow	main()	Uses CrewRunRequest, PlanApproval
scripts/run_api.py	API launcher wrapper	imports run() from API main	N/A
notebooks/project_20_enterprise_multi_agent_platform.py	Jupyter notebook deliverable	None	N/A
tests/	Unit/integration-style checks	test_planner.py, test_persistence.py, test_reports.py, etc.	Uses config + APIs as contracts

Module 3: Core Execution Flows

This module walks through the main operational paths and the real input/output data shapes.

Flow A: API planning and execution (/crew -> approve -> execute)

Entry file: src/crew_platform/api/main.py

Step-by-step: 1. POST /crew accepts CrewRunRequest. 2. Handler calls CollaborationService.create_plan(). 3. PlannerService.build_plan() creates PlanProposal with run_id and tasks. 4. Plan is stored by PersistenceStore.save_plan(). 5. POST /crew/{run_id}/approve sets approval state via apply_approval(). 6. POST /crew/{run_id}/execute calls execute_run(). 7. DAGExecutor.execute() runs dependency-ready tasks. 8. Task outputs are persisted (save_task), verified (VerificationService.verify()), optionally consensus-scored, then report is generated and persisted. 9. Final response is CrewRunResult serialized with model_dump(mode="json").

Key input shape:

```
{
  "query": "Create enterprise launch brief for AI multi-agent collaboration platform",
  "session_id": "default",
  "auto_execute": false,
```

```

    "force_consensus": false
}
Key plan task shape (TaskSpec):
{
  "task_id": "research",
  "title": "Collect evidence",
  "description": "Gather external and internal evidence relevant to objective",
  "agent_role": "Market Research Analyst",
  "dependencies": ["plan_scope"],
  "tools": ["web_search", "wikipedia", "url_fetcher"],
  "output_schema": "research_note",
  "retries_allowed": 2
}

```

Inside each executed task (`TaskExecution.result`), executor merges outputs as:

```

{
  "role": "Market Research Analyst",
  "content": "...LLM output...",
  "crewai_output": "...CrewAI output or fallback message...",
  "tools_used": ["web_search", "wikipedia", "url_fetcher"],
  "schema": "research_note"
}

```

Flow B: Planner internals (LLM-first, deterministic fallback)

File: `src/crew_platform/orchestration/planner.py`

Actual strategy: 1. `build_plan(query)` creates `run_id = "run-" + uuid hex prefix`. 2. Calls `_try_llm_plan(query)` with strict JSON prompt contract. 3. If parse/LLM fails, `_fallback_plan(query)` returns deterministic 7-task chain: - `plan_scope` -> `research` -> `analysis` -> `draft` -> `fact_check` -> `qa` -> `final_report` 4. For each task role, `AgentCatalog.ensure_role()` normalizes role and supports dynamic role creation for unknown roles. 5. Returns `PlanProposal` with assumptions list.

Planner prompt contract snippet (real function `_planner_prompt`):

```
"Output schema: {\"tasks\": [{\"task_id\":...,\"agent_role\":...,\"dependencies\":...}]}"
```

Flow C: DAG executor and retries

File: `src/crew_platform/orchestration/executor.py`

Execution model: 1. Build dependency map from task list. 2. Repeatedly select ready tasks (`all(dep in completed for dep in deps)`). 3. Run ready tasks concurrently with `asyncio.Semaphore(max_parallel_tasks)`. 4. Retry each

task up to `retries_allowed` with sleep backoff. 5. If no ready tasks while unfinished remain, mark deadlock failures.

Per-task internals in `_execute_single_task()`: - Build prompt from objective + agent goal + dependency context + tool context. - Call `OllamaProvider.generate()`. - Optionally call `CrewAIRunner.run_task()` if `use_crewai_execution` is true. - Merge into result dict. - Raise `RuntimeError("Empty agent output")` only if both content and `crewai_output` are empty.

Flow D: Verification and optional consensus

Files: - `src/crew_platform/orchestration/verification.py` - `src/crew_platform/orchestration/consensus.py`

Verification logic: - `factual_score`: references heuristic (`http`, `source`, `citation`). - `qa_score`: completed ratio + no-error ratio. - `reflection_score`: asks reflection model for 0..1; if parse fails, uses heuristic fallback. - `confidence = (factual + qa + reflection) / 3`. - `needs_rerun = confidence < settings.orchestration.consensus_trigger_confidence`.

Consensus logic: - Runs up to first 3 models from `settings.llm.consensus_models`. - Scores answers by length plus evidence/risk keywords. - Returns `ConsensusResult(selected_answer, candidate_answers, rationale)`.

Flow E: Report generation and export

File: `src/crew_platform/reports/generator.py`

What happens: 1. `generate()` builds `ReportArtifact` from tasks + verification. 2. Writes required formats from `settings.reports.always_formats` (default markdown + json). 3. On demand, exports html or pdf via `generate_on_demand()`. 4. `/reports/{run_id}/export` API endpoint triggers this flow.

`ReportArtifact` shape:

```
{
  "run_id": "run-...",
  "title": "Crew Platform Report - run-...",
  "generated_at": "ISO8601",
  "summary": "Objective... Completed tasks... Confidence...",
  "sections": [{"title": "task_id - role", "content": "...", "status": "completed", "confidence": 0.762},
  "references": ["http://..."],
  "metadata": {"objective": "...", "task_count": 7, "issues": []}]
}
```

Flow F: Knowledge ingestion and retrieval (RAG)

Files: - `src/crew_platform/rag/ingest.py` - `src/crew_platform/rag/retriever.py`

Ingestion: - POST /knowledge with either path or url. - ingest_path() reads local document by suffix (md/txt/pdf/csv/json/...), chunks text, stores chunks through RuntimeMemory.remember_run(). - ingest_url() downloads + extracts content, chunks, stores similarly.

Retrieval: - GET /knowledge?q=...&top_k=... - RAGRetriever.retrieve() -> RuntimeMemory.search() -> Chroma results.

Flow G: Tool invocation path

Files: - src/crew_platform/tools/registry.py - src/crew_platform/tools/factory.py

Lifecycle: 1. Tools are created and registered in create_default_registry().
2. Execution path calls ToolRegistry.invoke(name, payload, run_id).
3. Registry validates payload against tool input_model. 4. Tool runs asynchronously and output is validated against output_model. 5. Metrics + JSONL trace are recorded.

Example concrete tool contracts: - duckduckgo_search (tools/search.py)
- Input: { "query": str, "max_results": int } - Output: { "query": str, "results": [{ "title": str, "href": str, "body": str }] }
- calculator (tools/calculator.py) - Input: { "expression": str } - Output: { "value": float, "normalized_expression": str } -
memory_search (tools/memory_search.py -> SemanticSearchTool) - Input: { "query": str, "top_k": int } - Output: { "matches": [{ "id": str, "text": str, "distance": float }] }

Module 4: Setup & Run Guide

This section documents setup and run commands defined by repository files.

1) Prerequisites

- Python >=3.12,<3.13 (from pyproject.toml).
- uv package manager.
- Local Ollama server reachable at llm.base_url (default http://127.0.0.1:11434).

2) Install dependencies

```
uv venv .venv
source .venv/bin/activate
uv sync --all-groups --python 3.12
```

3) Configuration files to understand first

- configs/settings.yaml
- configs/agents.yaml

High-impact settings keys: - llm.default_model, llm.planner_model, llm.reflection_model, llm.consensus_models - orchestration.max_parallel_tasks, orchestration.default_retry_limit, orchestration.use_crewai_execution - memory.sqlite_path, memory.chroma_path - reports.output_dir, reports.always_formats, reports.on_demand_formats - tools.enabled_tools, tools.optional_tools, tools.enable_python_tool

4) Environment variables (.env keys)

From static code inspection, no required .env file is enforced.

Optional environment variables used by code: - CREW_PLATFORM_API_URL (Streamlit default API URL) - CREW_PLATFORM_DISABLE_CHROMA (disable semantic memory backend) - AGENT_OFFLINE_MODE (force network tools to offline fallback) - STREAMLIT_BROWSER_GATHER_USAGE_STATS (set to false in app.py launcher) - CHROMA_CACHE_DIR (set automatically by ChromaSemanticStore) - CREWAI_TRACING_ENABLED, CREWAI_DISABLE_TELEMETRY, CREWAI_DISABLE_TRACKING, OTEL_SDK_DISABLED (set automatically in CrewAIRunner)

5) Main startup commands (from entrypoints/scripts)

API:

```
uv run crew-platform-api
# or
uv run python scripts/run_api.py
```

CLI:

```
uv run crew-platform run "Your objective"
uv run crew-platform agents
uv run crew-platform doctor --api-url http://127.0.0.1:8000
```

Streamlit dashboard:

```
uv run python app.py
# or
uv run crew-platform dashboard
```

Programmatic demo script:

```
uv run python scripts/run_demo.py
```

6) Database migrations or seeding

No external migration framework is wired in this codebase.

Observed behavior from code: - SQLite schema is auto-created by Base.metadata.create_all(self.engine) in PersistenceStore.__init__.
- Chroma collection is auto-created by client.get_or_create_collection(...)

in `ChromaSemanticStore.__init__`. - There is no explicit seed script; data is produced by normal workflow execution.

Module 5: Study Plan & Practice Exercises

Ordered study plan

1. Read `pyproject.toml`, `configs/settings.yaml`, and `configs/agents.yaml`.
2. Read data contracts in `src/crew_platform/orchestration/models.py`.
3. Read orchestration core in this order:
 - `planner.py`
 - `executor.py`
 - `verification.py`
 - `consensus.py`
 - `service.py`
4. Read interfaces:
 - `api/main.py`
 - `cli/main.py`
 - `streamlit_app/`
5. Read infrastructure layers:
 - `llm/ollama.py`
 - `tools/base.py`, `tools/registry.py`, `tools/factory.py`
 - `memory/persistence.py`, `memory/runtime.py`
6. Read output and analytics:
 - `reports/generator.py`
 - `analytics/service.py`
 - `monitoring/service.py`
7. Finish with `tests/` to validate your understanding of expected behavior.

Practice exercises (with solution outlines)

1. Exercise: Trace what happens after `POST /crew`.
 - Task: List function calls in order until the plan is persisted.
 - Solution outline: `/crew` handler -> `_service().create_plan()` -> `planner.build_plan()` -> create `CrewRunResult` + `runs[run_id]` -> `persistence.save_plan(...)`.
2. Exercise: Explain the difference between `TaskSpec` and `TaskExecution`.
 - Task: Identify extra runtime fields.
 - Solution outline: `TaskExecution` extends `TaskSpec` with `status`, `timestamps`, `attempt`, `result`, `error`, `confidence`.
3. Exercise: Find where parallelism and retries are controlled.
 - Task: Name exact config keys and code locations.
 - Solution outline: `orchestration.max_parallel_tasks` and `orchestration.retry_backoff_seconds` used in `DAGExecutor.execute()`. `retries_allowed` comes from each

TaskSpec.

4. Exercise: Explain when consensus runs.
 - Task: State exact condition from code.
 - Solution outline: In `execute_run()`, consensus runs if `force_consensus` is `true` OR `consensus_enabled` and `verification.confidence < consensus_trigger_confidence`.
5. Exercise: Inspect tool safety for local files.
 - Task: Explain how path traversal is prevented.
 - Solution outline: `FileReaderTool`, `MarkdownReaderTool`, `PDFReaderTool`, etc. resolve path and reject if target is outside `workspace_root` (if `workspace_root` not in `path.parents` and `path != workspace_root`).
6. Exercise: Explain semantic memory disable behavior.
 - Task: Identify env var and resulting behavior.
 - Solution outline: `CREW_PLATFORM_DISABLE_CHROMA` in `RuntimeMemory.__init__`; when true, `semantic_store = None`, so `search()` returns empty list and semantic adds are skipped.
7. Exercise: Describe `ReportGenerator` export formats.
 - Task: List always-generated and on-demand formats from config and code.
 - Solution outline: `always_formats` defaults to `markdown/json`; on-demand supports `html/pdf/markdown/json` via `generate_on_demand()`.
8. Exercise: Add a new optional tool in your mind (no code changes required).
 - Task: Which files must change if you wanted a new built-in optional tool?
 - Solution outline: create tool class in `src/crew_platform/tools/`, then register in `tools/factory.py` gated by `optional_tools`, and include name in `configs/settings.yaml` optional list if desired.
9. Exercise: Explain approval gating.
 - Task: What happens if a run is executed without approval?
 - Solution outline: `execute_run()` checks `plan_approval_required` and `record.approved`; if not approved, status remains `AWAITING_APPROVAL` and `error` is set.
10. Exercise: Explain MCP internal call flow.
 - Task: Trace `/mcp/call` to actual tool output.
 - Solution outline: `/mcp/call` -> `InternalMCPServer.call_tool()` -> `ToolRegistry.invoke()` -> validated tool output returned as `MCPCallResponse`.

Understanding Checklist

Use this checklist to self-evaluate before contributing:

- I can explain how `CrewRunRequest` becomes a persisted `PlanProposal`.
- I can describe each field in `TaskExecution` and when it is populated.
- I can trace task dependency resolution in `DAGExecutor.execute()`.
- I can explain why a run may return `needs_rerun=true` in verification.
- I can explain the consensus trigger condition and selected models.
- I can describe where tool invocation is validated and logged.
- I can explain where `SQLite` and `Chroma` are initialized and how they degrade gracefully.
- I can list all implemented API endpoints and their purpose.
- I can export a report in `html` or `pdf` through the report endpoint or CLI command.
- I can identify where to add a new agent role or tool without breaking core orchestration.